

Project Handbook

Everything the system does, why it was built that way, and how to answer for it

This document is written for somebody who has never seen this project. By the end of it you should be able to run the system, explain every number on its screens, and answer a panel that asks why any of it was built the way it was.

It is a dealer console for rented heavy equipment. A rental company hands machines to customers and then loses sight of them - where they are, whether they are working, when they are due back. That gap costs money on both sides: the dealer bills for a machine doing nothing, and the customer pays for it. This system reads the telemetry those machines already emit and turns it into the four decisions a dealer actually makes.

THE FOUR QUESTIONS IT ANSWERS

- 1. Where is my money leaking?** Eight rules of three different kinds, each flag carrying the field it read and the threshold it crossed.
- 2. Can I commit this machine?** A ranking across the whole fleet by when each machine is genuinely free, answered with a confidence.
- 3. What is about to break?** A rolling mean and a least-squares trend, resolving to a real SAE J1939 fault code and the days of operation left.
- 4. What did acting on it save?** A ledger that keeps three different kinds of money apart instead of blending them into one flattering number.

Team	MBBS - Prajwal Patil, Neerav Babel, Tanish Balwal L, Mahantappa R Patted
Event	Caterpillar hackathon, Jain University
Stack	FastAPI / Python 3.11 - React 18 / TypeScript / Vite - PostgreSQL via Supabase
Scale	27 machines - 15,144 telemetry snapshots - 199 seeded events - 243 SKUs - 83 branches
Tests	62 passing, pinning every rule, every rupee and every confidence

PART ONE

Running it in five minutes

Two processes. The backend serves the data and does all the thinking; the frontend is three consoles that read it.

The backend

```
cd starter/api
ADMIN_TOKEN='<the dealer access key>' \
DATABASE_URL='postgresql://...supabase.com:5432/postgres' \
python -m uvicorn main:app --port 8000
```

Both variables are optional. Without DATABASE_URL it reads the JSON seed files and works identically. Without ADMIN_TOKEN the two destructive routes are unguarded, which is fine locally.

The frontend

```
cd starter/web
npm install
npm run dev # http://localhost:5199
npm run dev:phone # https on the LAN, for the QR scanner
```

Check it worked

```
curl -s localhost:8000/health
{"ok":true,"now":"2025-05-12","assets":27,"telemetry_snapshots":15144,...}
```

Interactive API documentation is live at localhost:8000/docs - every route, with a Try it out button.

Signing in

The console has three roles and each lands on a different screen. This is not cosmetic: showing a customer the dealer's exposure ledger would be showing them another customer's numbers.

Role	Sees	Needs
Viewer (a customer)	Only the machines on hire to their site, what they are being billed, and what to do next	A site, e.g. S003
Yard supervisor	What is ready to go out, what is late, what is coming back, what needs service. Check-in and check-out.	Nothing
Operations lead	The full board - briefing, forecast, ledger, map, rate card	The dealer access key

PART TWO

The shape of the system

```
BROWSER React 18 - TypeScript - Vite - TanStack Query
| three consoles behind one sign-in
| talks only to our API, never to the database
v
API FastAPI - Pydantic v2 - Python 3.11
| routes - validation - status projection - idempotency
|
+-----> intelligence.py pure functions. no I/O, no clock, no network
| analyze(assets, telemetry, bookings, config) -> bundle
|
+-----> store.py the ONE module that touches storage
DATABASE_URL set -> Supabase (PostgreSQL)
DATABASE_URL unset -> JSON seed files
```

Five decisions that shape everything else

Status is never stored

What: There is no status column. ACTIVE, IDLE, OVERDUE, UNASSIGNED and AT_YARD are computed from the event log every time anybody asks. Six of the fields on GET /assets exist nowhere in storage.

Why: A stored status is a second source of truth, and it drifts. The classic rental-system bug is a machine marked ACTIVE that has been in the yard a week because one update failed. Here that state cannot exist.

The event log is append-only

What: Check-out, assign, usage, condition and check-in each write one immutable row carrying who, what, where and when. Nothing is ever modified or deleted.

Why: The audit trail is the product. A dealer disputing a bill needs history, not current state - and you cannot recompute status from rows that have been overwritten.

Not one numeric literal in the rules

What: Every threshold, rate and window is a key in config.py - 34 of them - passed into analyze() as a dictionary. PUT /config changes them at runtime.

Why: A threshold buried in code is an opinion nobody can argue with. Change the excavator day rate on the Settings screen and every figure on the board moves, live.

The intelligence layer is a pure function

What: analyze(assets, telemetry, bookings, config). Same inputs, same outputs, always. It cannot read the clock - the date arrives in the config, pinned to 2025-05-12.

Why: It makes the engine testable and the demo reproducible. A function that reads the real clock gives a different answer tomorrow and cannot be asserted on.

One module touches storage

What: The API asks `store.read(name)` and gets a list of records. Whether they came from PostgreSQL or off the disk is that module's business and nobody else's.

Why: We moved from JSON to PostgreSQL mid-build by changing one function, and nothing above it noticed. That is what the seam was for.

How it decides

The single most important thing to understand, and to say out loud: **almost nothing here is a machine-learning model, and that is a decision rather than a shortcut.** On seven given rows nothing can be validated statistically. Determinism and auditability can.

Anomaly detection - eight rules, three kinds

Not one detector with a threshold. Three genuinely different kinds of detection. Every flag ships the field names it read, the values it saw and the threshold it crossed - so a dealer who disagrees can see exactly which number to argue about.

Rule	Kind	Fires when	Severity	Money bucket
R1	cross-field	On rent with no site, but hours are being burned	CRITICAL	waste
R2	threshold	Utilisation under the idle threshold for enough days	CRITICAL	waste
R3	threshold	Zero engine hours across the whole rental	CRITICAL	waste
R4	contradiction	Operating days exceed the rental window - the record cannot be true	WARNING	recoverable
R5	threshold	Hours since service at or past 200	WARNING	avoided
R6	threshold	Still out past the scheduled return date	CRITICAL	recoverable
R7	cross-field	On rent with nobody assigned to run it	WARNING	waste
R8	threshold	Due back inside the reminder window	INFO	reminder - zero

Two of these are worth pointing at deliberately

R4 finds bad data, not bad behaviour. It does not compare a number to a threshold - it compares two fields that cannot both be true. A machine cannot have operated for more days than it was rented for. We demonstrate it catching genuine defects in the 806,485-row catalogue we did not author: three rows priced below cost, 174 duplicate invoice IDs. Finding errors in someone else's data is a far stronger claim than finding them in your own.

R8 is deliberately worth zero. A due-back reminder fires, is shown, and is priced at nothing, because nothing has been lost yet. Pricing it would inflate the headline with money that was never at risk.

Maintenance risk - the one statistical model

This is the only place in the system where a statistical method is used. Being precise about that boundary is the strongest thing we can say about our own AI.

```

rolling = np.mean(last 24 coolant readings)
slope = np.polyfit(seconds, temps, 1)[0] # least squares, degree 1
over a 7-day trailing window

fire when rolling > 105 C AND slope > 0.5 C/day

days_to_failure = (115 C - rolling) / slope in OPERATING days
-----
EQX1005 111.54 C rising 0.801 C/day -> 4.33 days
emits SPN 110 / FMI 0 - cooling package: radiator core + thermostat

```

Why not a model? A model needs labelled failures to learn from and we have zero - nothing in this dataset records a machine that actually failed. Any classifier would be fitting noise and calling it intelligence. A trend line on a physical quantity that genuinely rises before a cooling failure is defensible, and a judge can check the arithmetic on screen.

Why operating days? A machine that is not running is not heating up, so the countdown pauses while it sits in the yard. This is the detail that shows the model understands the machine rather than the spreadsheet.

SPN 110 / FMI 0 is a genuine SAE J1939 fault code - engine coolant temperature, data valid but above normal operating range. We emit the real code.

Availability - a commitment, not a probability

```

rank every machine of the requested type by, in order:
1 transfer local before a branch transfer
2 free_from earliest genuinely available
3 condition_grade A before B before C
4 cumulative_operating_hours the least-worn of equals

confidence at yard 1.00 - 2+ days early 0.90 - day before 0.75 - same day 0.50
minus 0.15 if it must move between branches

```

A dealer cannot act on a probability; they need a machine ID and a date. The best answer in this data is EQX1007 at confidence 1.00 - "you do not have to wait for Friday, you already have one doing nothing." The machine we flagged as waste is the one that solves the request.

Demand - projected, not fitted

```

Excavator units by month, eleven years of catalogue:
3433 3241 3409 3304 3466 3377 3485 3407 3326 3355 3427 3553
flat to within 3%

So we project mechanically instead:
a site is working a type at a rate read off the machine's own cumulative
counter, and a machine of that type is booked to leave inside the horizon
-> the site is short from the day it goes

four config values are the entire model:
horizon 7d - window 14d - floor 1.0 h/day - full confidence at 4.0 h/day
confidence = min(1.0, lost_rate / 4.0) one division, nothing hidden

```

Fit a model to a flat series and you get a horizontal line with a confidence interval drawn on it. That is decoration, not a forecast. A **booking is never blended with a projection** - a request somebody actually made is not a guess, so it is labelled and reported at full confidence. And the cover for every row comes from the availability engine rather than a second calculation, so the two panels cannot drift apart.

The money, and how every rupee is derived

already burned INR 12,66,470 spent on machines producing nothing
 still billable INR 21,61,000 overdue rental you can invoice today
 downtime avoided INR 18,000 one service day EQX1005 will not lose

 open exposure INR 34,45,470

These are three different kinds of money and the model refuses to blend them. Waste has already left the building. Billable is recoverable if somebody makes a phone call. Avoided has not happened yet. A single INR 34.4 lakh "savings" headline is larger and worse: the first judge who asks "is that money you lost or money you might get?" collapses it.

The idle formula

```
hourly_rate = day_rate / (engine_hours_day + idle_hours_day)
idle_waste = idle_hours_day * hourly_rate * operating_days
```

The rate is divided by the machine's **actual** working day, not a nominal eight hours. A machine billed at INR 15,000 that is on for 11.5 hours costs INR 1,304.35 an hour; ten of those hours produce nothing. That is a defensible unit cost rather than a chosen one.

The de-duplication rule - the most important line in the model

Machine	Rule	What it saw	Value
EQX1002	R1	zero output for 20 operating days	INR 4,40,000
EQX1002	R3	on rent, attached to no site	INR 4,40,000
EQX1002	R7	no operator (half the rental line)	INR 2,20,000
summed		<i>what a naive model would claim</i>	INR 11,00,000
counted		<i>largest rule only</i>	INR 4,40,000

Within a bucket, a machine counts once, at its largest firing rule. Three rules firing on one rental line is three reasons, not three invoices. Say this before a judge asks for it - volunteering it is the single most credible move in the whole pitch.

The rate card, and the second one underneath it

Type	Published card (default)	Price-implied (cross-check)
Excavator	INR 15,000	INR 15,000
Bulldozer	INR 18,000	INR 13,000
Crane	INR 22,000	INR 18,000
Grader	INR 12,000	INR 19,500

The second column is derived from the 243-SKU catalogue: median category SKU price times k, with k = 2.131 anchored so the excavator reproduces the published rate. It proves the published card is not invented. **Grader**

is the outlier and we do not hide it - there is no grader category in the catalogue, so RoadPavers is the nearest real match and the gap is stated rather than smoothed.

PART FIVE

Where the data comes from

This is not Caterpillar's dataset, and we will not claim it is. It is a public distribution catalogue: 163 brands, of which Caterpillar is 4.99% of rows. That is provable from a committed manifest carrying the file's SHA-256. One Volvo invoice appearing on screen after claiming otherwise would end the credibility of everything else.

What	Where it comes from
The seven given assets	Every numeric value kept exactly as supplied. Only invented metadata - model, serial number, site - was replaced with real catalogue values.
Twenty more machines	Generated from real SKUs at the four Indian branches with a seeded RNG, so the generator is reproducible run to run.
Telemetry	15,144 snapshots, ISO 15143-3 / AEMP 2.0 field names, reconciling exactly to each machine's engine and idle hours per day.
Catalogue	243 SKUs, 83 branches across 36 countries, 236 operator customers - extracted once from the 806,485-row source file.
Day rates	Asserted from the published card. The price-implied column is the cross-check, not the default.
Transit times	A stated assumption, printed on the Settings screen and editable. NOT from the dataset - see below.

Why we refused the dataset's own delivery times. Its DeliveryDate lead time is order-to-delivery for a purchase - median 15 days, identical across all four Indian branches. Wire that in as transit and nothing is ever committable. Knowing which columns to reject is as much a part of the work as using the rest.

If the panel asks

Prepared answers, not improvised ones. Each is honest about a limit, because the honest version is stronger than the flattering one and a panel will find the seam anyway.

Where is the machine learning? This looks like if-statements.

"One least-squares slope, and a language model that only phrases what the rules computed. Everything else is a rule, on purpose. We have seven given rows and zero recorded failures. Any classifier trained on that is fitting noise and calling it intelligence. What we can prove instead is determinism: 62 tests pin every rupee, and every flag names the field, the value and the threshold it crossed."

Then turn it: "Which would you rather defend to a dealer - an accuracy figure from seven rows, or a number they can click into and check?"

Why no demand forecasting? It was on the brief.

"Because the data has no demand signal, and we can show you. Eleven years of monthly volume, flat to within three percent. Fit a model to that and you get a horizontal line with a confidence interval on it. We built the prediction the data actually supports - a mechanical projection off each machine's own counter - and it produces your own example unprompted: S003 needs an excavator, and the machine to send is EQX1007."

Is this Caterpillar's dataset?

"No, and we will not claim it is. It is a public distribution catalogue - 163 brands, Caterpillar is 4.99% of rows. We used it for SKUs, branches and real price ratios only. That is provable from a committed manifest with the file's SHA-256 in it."

What is your accuracy?

"On seven rows, nobody's accuracy figure is real. Ours is deterministic and auditable instead. The same inputs always give the same outputs - the intelligence layer cannot even read the clock, the date arrives in the config - and 62 tests assert every anomaly, every rupee and every confidence."

Why FastAPI? Why not Postgres from day one?

"FastAPI because Pydantic makes the schema the contract - four of us built against a frozen schemas.py in parallel, and a malformed request never reaches any logic. And we did move to PostgreSQL; it is on Supabase now. It took changing one function, because only one module has ever touched storage."

Does this scale?

"The telemetry contract is ISO 15143-3, the events are append-only, and status is a projection rather than a stored column - so history is never destroyed by an update. The catalogue extraction already runs over 806,485 rows. At your volume this is a stream into a warehouse; the seams are already where they need to be."

Your seven-asset utilisation was 39.2%. Now it says 71.3%. Which is it?

"Both, and the difference is the point. 39.2% is the seven rows you gave us. 71.3% is the twenty-seven-machine fleet we grounded in the catalogue around them. We quote the seven when we are talking about your data and the fleet when we are talking about the board."

How do we know the synthetic history is not just made up?

"Because it reconciles to your rows exactly. Every generated machine satisfies $\text{cumulative_operating_hours} = \text{engine_hours_day} \times \text{operating_days}$, and the seven you gave us keep every numeric value untouched. The generator is seeded, so it is reproducible."

Is that real authentication?

"It is a real enforcement boundary and an honest audit trail. It is not identity - and it is one function away from being identity. The name is self-declared, so the log is attributable rather than authenticated. Swap `require_admin` for an OIDC check against the dealer's own identity provider and the actor comes from the verified token. Nothing downstream moves, because every consumer already reads the actor off the event log."

Why is there no password?

"Because a hand-rolled password store beside the seed data would have been the largest attack surface in this project and the weakest thing in it. We built the part that is real - a server-checked key on the destructive routes - and were honest about the part that is not."

PART SEVEN

What we deliberately did not build

Stated so nobody on the team is tempted to over-claim mid-answer. Knowing where your system stops is part of engineering it.

Not built	Because
A demand regression	No signal to fit - eleven years flat to within 3%.
A failure classifier	Zero labelled failures anywhere in the data.
Transit times from the dataset	Its lead time measures purchase delivery, not transit, and is identical across all four Indian branches.
Blended money totals	Waste, billable and avoided stay apart in every sentence.
A language model in the briefing	Those six sentences are arithmetic. No model is involved and every figure appears elsewhere on the screen.
Password accounts	It would have been the largest attack surface and the weakest thing in the project.

PART EIGHT

Where to find things

Path	What is in it
api/intelligence.py	The engine. All eight rules, the maintenance model, the availability ranking, the demand projection, the value summary. Pure functions.
api/config.py	Every threshold, rate and window. 34 keys. Change anything here and the whole board moves.
api/main.py	Routes, validation, status projection, idempotency, the admin guard, the assistant endpoint.
api/store.py	The only module that touches storage. Reads Supabase or JSON; writes back to Supabase without ever failing the request.
api/schemas.py	The frozen contract. Four people built against this in parallel.
db/schema.sql	Ten tables and two views. The views project status in SQL exactly as Python does.
db/load_supabase.py	Loads everything into PostgreSQL. Idempotent.
db/restore_baseline.py	Puts the demo back to its opening numbers after a rehearsal. Run this before presenting.
data/	Seed files plus the catalogue extracted from the source CSV, and source_manifest.json which fingerprints it.
web/src/pages/	The three consoles: CustomerView, YardBoard, FleetBoard - plus SignIn, Scan, AssetPanel, Settings.
pitch/	qa_prep.md, value_model.md, demo_script.md - the written answers behind every claim.

Before you present

Rehearsing the demo spends it. Every check-in writes to the database and stays written, so a practice run moves the headline figures off the numbers printed in this document.

Restore, restart, and verify the board reads **9 things need you today** and **INR 34,45,470** before you walk in:

```
python db/restore_baseline.py
```

The three sentences that survive if you are cut short

"Every number on this screen can be traced to the field that produced it."

"We separate money already burned from money still billable from downtime avoided, because adding them gives you a headline that does not survive a question."

"We did not build a demand forecast, and we can show you the eleven years of flat data that is the reason why."